In [32]:
```python
import json
import os
import re
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import matplotlib.pyplot as plt
from tqdm import tqdm
```

导入相关包

In [33]:
```python
# 数据路径
DATA_DIR = "./poetry_data"
DATA_FILES = ["poet.song.40000.json", "poet.song.41000.json",
              "poet.song.42000.json", "poet.song.43000.json"]

# 生成格式配置（七言绝句：4句，每句7字）
POEM_TYPE = "七言绝句"
SENTENCE_NUM = 4    # 诗句总数量
SENTENCE_LEN = 7    # 每句字数
TOTAL_LEN = SENTENCE_NUM * SENTENCE_LEN  # 全诗总字数

# 模型超参数
EMBEDDING_DIM = 256
HIDDEN_DIM = 512
NUM_LAYERS = 2
DROPOUT = 0.2
BATCH_SIZE = 64
EPOCHS = 20
LEARNING_RATE = 0.001
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# 生成配置
START_WORDS = "明月"  # 起始词
GENERATE_TEMPERATURE = 0.7  # 温度系数，越小越保守，越大越随机
```

设置参数

In [34]:
```python
def load_poems():
    """加载并筛选符合格式的古诗（修复中文过滤和诗句拆分问题）"""
    poems = []
    # 正则表达式：只保留中文字符，去掉所有标点、空格、数字等
    chinese_only = re.compile(r'[^\u4e00-\u9fa5]')

    for file in DATA_FILES:
        file_path = os.path.join(DATA_DIR, file)
        with open(file_path, "r", encoding="utf-8") as f:
            data = json.load(f)
            for item in data:
                paragraphs = item["paragraphs"]
                all_sentences = []
                valid = True

                for para in paragraphs:
                    # 只保留中文字符
```

```
                    clean_para = chinese_only.sub('', para)
                    # 每个段落必须是2句×7字=14字（数据集格式）
                    if len(clean_para) != 2 * SENTENCE_LEN:
                        valid = False
                        break
                    # 拆分成两句
                    sentence1 = clean_para[:SENTENCE_LEN]
                    sentence2 = clean_para[SENTENCE_LEN:]
                    all_sentences.append(sentence1)
                    all_sentences.append(sentence2)

                # 筛选：总句数正好等于要求的数量
                if valid and len(all_sentences) == SENTENCE_NUM:
                    full_poem = "".join(all_sentences)
                    poems.append(full_poem)

    print(f"共加载符合{POEM_TYPE}格式的古诗：{len(poems)}首")
    return poems

def build_vocab(poems):
    """构建词汇表"""
    all_chars = set()
    for poem in poems:
        all_chars.update(poem)
    # 添加特殊字符
    all_chars = sorted(list(all_chars))
    char2idx = {char: idx+1 for idx, char in enumerate(all_chars)}  # 0留给paddi
    char2idx["<PAD>"] = 0
    idx2char = {idx: char for char, idx in char2idx.items()}
    vocab_size = len(char2idx)
    print(f"词汇表大小：{vocab_size}")
    return char2idx, idx2char, vocab_size

class PoetryDataset(Dataset):
    """古诗数据集类"""
    def __init__(self, poems, char2idx, seq_len):
        self.poems = poems
        self.char2idx = char2idx
        self.seq_len = seq_len
        self.data = self._prepare_data()

    def _prepare_data(self):
        """将古诗转换为数字序列"""
        sequences = []
        for poem in self.poems:
            # 转换为索引
            seq = [self.char2idx[c] for c in poem]
            # 生成输入和标签（输入：前n-1字，标签：后n-1字）
            for i in range(1, len(seq)):
                input_seq = seq[:i]
                target = seq[i]
                # 填充到固定长度
                input_seq = [0]*(self.seq_len - len(input_seq)) + input_seq
                sequences.append((input_seq, target))
        return sequences

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
```

```
        input_seq, target = self.data[idx]
        return torch.tensor(input_seq, dtype=torch.long), torch.tensor(target, d
```

数据预处理

In [35]:
```
class LSTMModel(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim, num_layers, dropou
        super(LSTMModel, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=0)
        self.lstm = nn.LSTM(embedding_dim, hidden_dim, num_layers,
                            batch_first=True, dropout=dropout)
        self.fc = nn.Linear(hidden_dim, vocab_size)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, hidden=None):
        # x: [batch_size, seq_len]
        embed = self.embedding(x)  # [batch_size, seq_len, embedding_dim]
        embed = self.dropout(embed)

        if hidden is None:
            output, hidden = self.lstm(embed)
        else:
            output, hidden = self.lstm(embed, hidden)

        # 取最后一个时间步的输出
        output = output[:, -1, :]  # [batch_size, hidden_dim]
        output = self.dropout(output)
        logits = self.fc(output)  # [batch_size, vocab_size]
        return logits, hidden
```

LTSM模型定义

In [36]:
```
def train_model(model, dataloader, criterion, optimizer, epochs, device):
    model.train()
    loss_history = []

    for epoch in range(epochs):
        total_loss = 0
        pbar = tqdm(dataloader, desc=f"Epoch [{epoch+1}/{epochs}]")

        for batch_idx, (inputs, targets) in enumerate(pbar):
            inputs = inputs.to(device)
            targets = targets.to(device)

            # 前向传播
            logits, _ = model(inputs)
            loss = criterion(logits, targets)

            # 反向传播与优化
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            total_loss += loss.item()
            pbar.set_postfix({"Loss": f"{loss.item():.4f}"})

            # 每200步打印一次Loss（和作业示例格式一致）
            if (batch_idx + 1) % 200 == 0:
                print(f"\nEpoch [{epoch+1}/{epochs}], Step [{batch_idx+1}/{len(d
```

```
        # 计算平均Loss
        avg_loss = total_loss / len(dataloader)
        loss_history.append(avg_loss)
        print(f"\n==== Epoch {epoch+1} Average Loss: {avg_loss:.4f} ====")

        # 每个epoch结束后生成一首示例古诗
        print("【生成演示】:")
        generate_poem(model, START_WORDS, char2idx, idx2char, device)
        print("-"*50)

    return loss_history
```

训练函数

In [37]:

```
def generate_poem(model, start_words, char2idx, idx2char, device, temperature=GE
    model.eval()
    generated = list(start_words)

    with torch.no_grad():
        # 初始化隐藏状态
        hidden = None

        # 先输入起始词
        for char in start_words:
            input_seq = torch.tensor([[char2idx[char]]], dtype=torch.long).to(de
            logits, hidden = model(input_seq, hidden)

        # 生成剩余字符
        for _ in range(TOTAL_LEN - len(start_words)):
            # 取最后一个字符作为输入
            last_char = generated[-1]
            input_seq = torch.tensor([[char2idx[last_char]]], dtype=torch.long).
            logits, hidden = model(input_seq, hidden)

            # 温度采样
            logits = logits / temperature
            probs = torch.softmax(logits, dim=1).cpu().numpy()[0]
            next_idx = np.random.choice(len(probs), p=probs)
            next_char = idx2char[next_idx]

            generated.append(next_char)

    # 按格式输出（每句7字，共4句）
    poem = "".join(generated)
    for i in range(SENTENCE_NUM):
        print(poem[i*SENTENCE_LEN : (i+1)*SENTENCE_LEN])
```

生成函数

In [38]:

```
def plot_loss(loss_history):
    plt.figure(figsize=(10, 6))
    plt.plot(range(1, len(loss_history) + 1), loss_history, 'b-', linewidth=2)
    plt.title('Training Loss Curve', fontsize=14)
    plt.xlabel('Epoch', fontsize=12)
    plt.ylabel('Train Loss', fontsize=12)
    plt.grid(True, linestyle='--', alpha=0.7)
```

```
    plt.savefig('training_loss.png', dpi=300, bbox_inches='tight')
    plt.show()
```

绘制曲线

In [39]:
```
if __name__ == "__main__":
    # 1. 加载并预处理数据
    print("正在加载数据...")
    poems = load_poems()
    if len(poems) == 0:
        raise ValueError("未找到符合格式的古诗，请检查数据文件或格式配置")

    char2idx, idx2char, vocab_size = build_vocab(poems)
    dataset = PoetryDataset(poems, char2idx, seq_len=TOTAL_LEN-1)
    dataloader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)

    # 2. 初始化模型
    print("正在初始化模型...")
    model = LSTMModel(vocab_size, EMBEDDING_DIM, HIDDEN_DIM, NUM_LAYERS, DROPOUT
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE)

    # 3. 训练模型
    print("开始训练...")
    loss_history = train_model(model, dataloader, criterion, optimizer, EPOCHS,

    # 4. 绘制Loss曲线
    print("绘制训练Loss曲线...")
    plot_loss(loss_history)

    # 5. 保存模型
    torch.save(model.state_dict(), "poetry_lstm_model.pth")
    print("模型已保存为 poetry_lstm_model.pth")

    # 6. 最终生成示例
    print("\n" + "="*30 + " 最终生成结果 " + "="*30)
    generate_poem(model, START_WORDS, char2idx, idx2char, DEVICE)
```

```
正在加载数据...
共加载符合七言绝句格式的古诗：902首
词汇表大小：2823
正在初始化模型...
开始训练...
Epoch [1/20]:  53%|█████▌      | 201/381 [00:23<00:20,  8.59it/s, Loss=7.0870]
Epoch [1/20], Step [200/381], Loss: 7.2270
Epoch [1/20]: 100%|██████████| 381/381 [00:44<00:00,  8.55it/s, Loss=7.1343]
==== Epoch 1 Average Loss: 7.1326 ====
【生成演示】:
明月穿領惟夜意
雨碧戎一時見去
枝五楚無人雲春
人似歸興似縑園
--------------------------------------------------
Epoch [2/20]:  53%|█████▌      | 201/381 [00:21<00:18,  9.49it/s, Loss=6.3436]
Epoch [2/20], Step [200/381], Loss: 6.7276
Epoch [2/20]: 100%|██████████| 381/381 [00:40<00:00,  9.51it/s, Loss=6.8501]
```

```
==== Epoch 2 Average Loss: 6.7787 ====
【生成演示】:
明月去來時不知
親豈知若人不足
遠隔有且來愛已
礙一我來亦愛中
--------------------------------------------------
Epoch [3/20]:  53%|█████▎     | 201/381 [00:21<00:18,  9.54it/s, Loss=6.1755]
Epoch [3/20], Step [200/381], Loss: 5.8883
Epoch [3/20]: 100%|██████████| 381/381 [00:40<00:00,  9.39it/s, Loss=5.7974]
==== Epoch 3 Average Loss: 5.9813 ====
【生成演示】:
明月四湘不知時
也遙時人間便是
青蠅松水只水流
盡空浮雲間日誰
--------------------------------------------------
Epoch [4/20]:  53%|█████▎     | 201/381 [00:21<00:19,  9.45it/s, Loss=4.5936]
Epoch [4/20], Step [200/381], Loss: 4.1588
Epoch [4/20]: 100%|██████████| 381/381 [00:40<00:00,  9.43it/s, Loss=4.4253]
==== Epoch 4 Average Loss: 4.4479 ====
【生成演示】:
明月明吾騷月無
人假賓酷梁紅出
嘆推武心更無情
驅假洗松明書眼
--------------------------------------------------
Epoch [5/20]:  53%|█████▎     | 201/381 [00:21<00:19,  9.39it/s, Loss=2.3620]
Epoch [5/20], Step [200/381], Loss: 2.6507
Epoch [5/20]: 100%|██████████| 381/381 [00:40<00:00,  9.40it/s, Loss=3.3800]
==== Epoch 5 Average Loss: 2.6535 ====
【生成演示】:
明月樽菰川盛聲
聞舊詩書本賢在
徘徊何樂息悟安
忠規綃當時事老
--------------------------------------------------
Epoch [6/20]:  53%|█████▎     | 201/381 [00:21<00:19,  9.29it/s, Loss=1.1102]
Epoch [6/20], Step [200/381], Loss: 1.2447
Epoch [6/20]: 100%|██████████| 381/381 [00:40<00:00,  9.43it/s, Loss=1.4936]
==== Epoch 6 Average Loss: 1.2110 ====
【生成演示】:
明月光竿正立清
風風草海秋色入
青蝶圖氣付已過
形堂更尋殘雪滿
--------------------------------------------------
Epoch [7/20]:  53%|█████▎     | 201/381 [00:21<00:19,  9.36it/s, Loss=0.3976]
Epoch [7/20], Step [200/381], Loss: 0.3393
Epoch [7/20]: 100%|██████████| 381/381 [00:40<00:00,  9.35it/s, Loss=0.6783]
==== Epoch 7 Average Loss: 0.4405 ====
【生成演示】:
明月樽洗寂寂山
穿寺似藏雲未卷
黃金勢熟榻溪來
洞一年明月夜深
--------------------------------------------------
```

```
Epoch [8/20]:  53%|█████▌      | 201/381 [00:21<00:19,  9.38it/s, Loss=0.0968]
Epoch [8/20], Step [200/381], Loss: 0.1749
Epoch [8/20]: 100%|██████████| 381/381 [00:40<00:00,  9.39it/s, Loss=0.3630]
==== Epoch 8 Average Loss: 0.1909 ====
【生成演示】:
明月滿庭無隱伏
夜深惟有此君知
大地只有殘頭見
土枝枝上月凝梅
--------------------------------------------------
Epoch [9/20]:  53%|█████▌      | 201/381 [00:21<00:19,  9.42it/s, Loss=0.1149]
Epoch [9/20], Step [200/381], Loss: 0.1050
Epoch [9/20]: 100%|██████████| 381/381 [00:40<00:00,  9.38it/s, Loss=0.0814]
==== Epoch 9 Average Loss: 0.1313 ====
【生成演示】:
明月洗洗如露死
遙想宴罷不知消
息處至今鄉老望
孤峰纏解不疑今
--------------------------------------------------
Epoch [10/20]:  53%|█████▌      | 201/381 [00:21<00:19,  9.46it/s, Loss=0.0250]
Epoch [10/20], Step [200/381], Loss: 0.1138
Epoch [10/20]: 100%|██████████| 381/381 [00:40<00:00,  9.45it/s, Loss=0.2779]
==== Epoch 10 Average Loss: 0.1135 ====
【生成演示】:
明月滿庭無隱伏
夜深惟有此君知
我得愁歸時恨隨
陽遙似君家只應
--------------------------------------------------
Epoch [11/20]:  53%|█████▌      | 201/381 [00:21<00:19,  9.42it/s, Loss=0.1248]
Epoch [11/20], Step [200/381], Loss: 0.1408
Epoch [11/20]: 100%|██████████| 381/381 [00:40<00:00,  9.41it/s, Loss=0.2119]
==== Epoch 11 Average Loss: 0.1061 ====
【生成演示】:
明月樽罍忘思此
身何似到山空駐
畫圖心種欲安堪
吟晚浪海邊子未
--------------------------------------------------
Epoch [12/20]:  53%|█████▌      | 201/381 [00:21<00:19,  9.46it/s, Loss=0.0203]
Epoch [12/20], Step [200/381], Loss: 0.1296
Epoch [12/20]: 100%|██████████| 381/381 [00:40<00:00,  9.45it/s, Loss=0.0141]
==== Epoch 12 Average Loss: 0.1001 ====
【生成演示】:
明月樽高露濕題
詩欲盡江南十十
里湖秋未歸海未
得却歸櫂飛飛簷
--------------------------------------------------
Epoch [13/20]:  53%|█████▌      | 201/381 [00:21<00:19,  9.47it/s, Loss=0.1543]
Epoch [13/20], Step [200/381], Loss: 0.1621
Epoch [13/20]: 100%|██████████| 381/381 [00:40<00:00,  9.40it/s, Loss=0.1254]
```

==== Epoch 13 Average Loss: 0.0960 ====
【生成演示】:
明月華圓滿滿庭
無隱伏春中回且
半壺桃李半花浮
半半中春雨霏一

Epoch [14/20]: 53%|█████▎ | 201/381 [00:21<00:19, 9.42it/s, Loss=0.0613]
Epoch [14/20], Step [200/381], Loss: 0.2382
Epoch [14/20]: 100%|██████████| 381/381 [00:40<00:00, 9.36it/s, Loss=0.0991]
==== Epoch 14 Average Loss: 0.0921 ====
【生成演示】:
明月華竿幾送無
用情喚明如何在
及公公魚輩不將
名傾屏且晴天淡

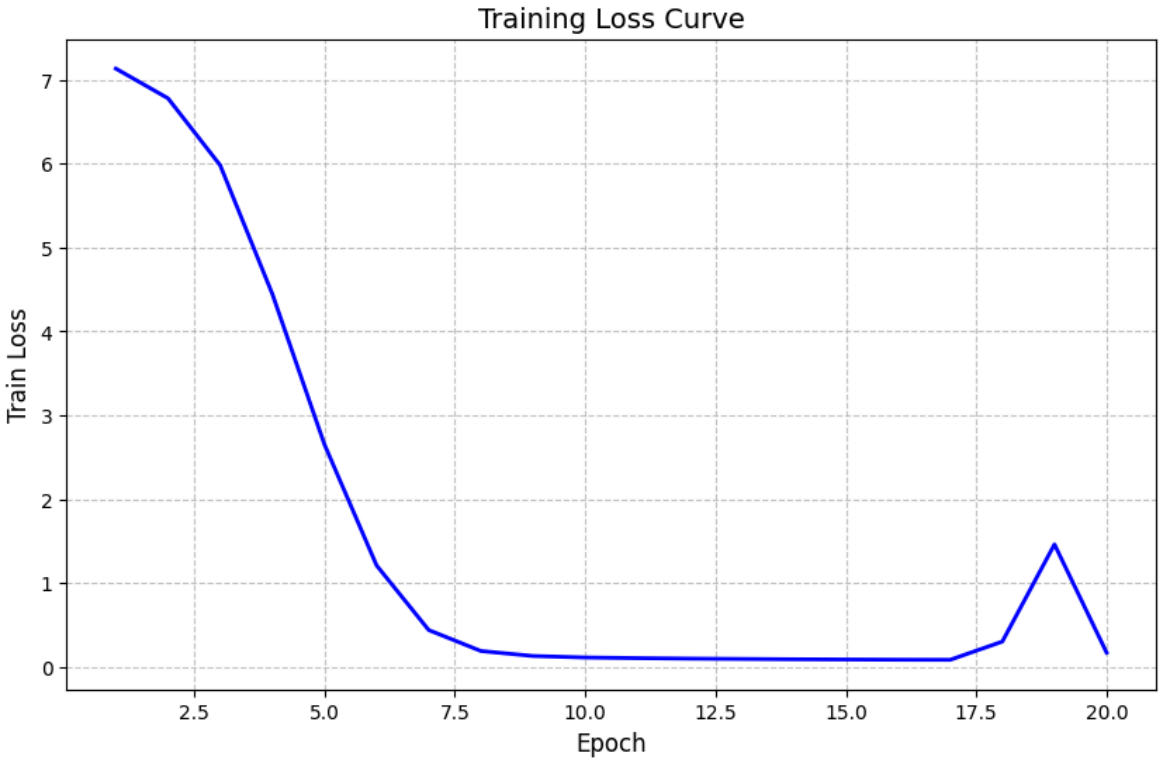
Epoch [15/20]: 53%|█████▎ | 201/381 [00:21<00:19, 9.33it/s, Loss=0.0642]
Epoch [15/20], Step [200/381], Loss: 0.0606
Epoch [15/20]: 100%|██████████| 381/381 [00:40<00:00, 9.41it/s, Loss=0.1404]
==== Epoch 15 Average Loss: 0.0895 ====
【生成演示】:
明月洗空爭露氣
凉時有酒醉來情
緒似秋烟水畔林
間不計年世事紛

Epoch [16/20]: 53%|█████▎ | 201/381 [00:21<00:19, 9.40it/s, Loss=0.0784]
Epoch [16/20], Step [200/381], Loss: 0.0103
Epoch [16/20]: 100%|██████████| 381/381 [00:40<00:00, 9.41it/s, Loss=0.2539]
==== Epoch 16 Average Loss: 0.0870 ====
【生成演示】:
明月墀繡屐想無
蹤庭戶新凉綠簟
空寄語金泥舊裙
衩莫教放盡石榴

Epoch [17/20]: 53%|█████▎ | 201/381 [00:21<00:19, 9.38it/s, Loss=0.0068]
Epoch [17/20], Step [200/381], Loss: 0.0478
Epoch [17/20]: 100%|██████████| 381/381 [00:40<00:00, 9.38it/s, Loss=0.0750]
==== Epoch 17 Average Loss: 0.0858 ====
【生成演示】:
明月粟郎省向江
頭山雪直湘華千
少年事都爲無情
旋祇憂元無一物

Epoch [18/20]: 53%|█████▎ | 201/381 [00:21<00:18, 9.50it/s, Loss=0.0257]
Epoch [18/20], Step [200/381], Loss: 0.0294
Epoch [18/20]: 100%|██████████| 381/381 [00:40<00:00, 9.43it/s, Loss=1.3952]
==== Epoch 18 Average Loss: 0.3029 ====
【生成演示】:
明月知時秋爲酒
勸人終不知禪暇
不向試酒任還由
得病餘竿春風客

```
Epoch [19/20]:  53%|█████▎      | 201/381 [00:21<00:19,  9.34it/s, Loss=1.8459]
Epoch [19/20], Step [200/381], Loss: 1.4439
Epoch [19/20]: 100%|██████████| 381/381 [00:40<00:00,  9.40it/s, Loss=1.1625]
==== Epoch 19 Average Loss: 1.4620 ====
【生成演示】:
明月華竿上秋容
太白蒼茫如卻乾
人在不是垂頭晝
卻恨歸時欲飽眠
--------------------------------------------------
Epoch [20/20]:  53%|█████▎      | 201/381 [00:21<00:19,  9.38it/s, Loss=0.1615]
Epoch [20/20], Step [200/381], Loss: 0.1313
Epoch [20/20]: 100%|██████████| 381/381 [00:40<00:00,  9.42it/s, Loss=0.3040]
==== Epoch 20 Average Loss: 0.1715 ====
【生成演示】:
明月讀離騷清詩
題柱幾年病骨影
清田變上病上春
力老困情多少宅
--------------------------------------------------
绘制训练Loss曲线...
```




```
模型已保存为 poetry_lstm_model.pth

============================== 最终生成结果 ==============================
明月粟還更向咸
陽游無盡似一枝
輕翠輕近天望花
隨柳過前川旁人
```

主函数